

BUILD A LARGE LANGUAGE MODEL

Capítulo 3 - Coding attention mechanisms (Codificação de mecanismos de atenção)

Anotações:

- Simplified self-attention: Introduz a ideia geral.
- Self-attention: Com pesos treináveis que formam a base do mecanismo usado na LLM.
- Causal attention: Permite o modelo considerar apenas entradas anteriores e atuais na sequência para garantir ordem temporal durante a geração de texto.
- Multi-head attention: Extensão do self-attention e causal attention, permite ao modelo atender informações ao mesmo tempo de diferentes representações de subespaços.
- O simplified self-attention é aplicado antes de adicionar pesos treináveis, o casual attention adiciona máscaras ao self-attention que permite a LLM gerar uma palavra por vez. Então o multi-head attention organiza o mecanismo de atenção em múltiplas cabeças, permitindo que o modelo capture vários aspectos dos dados de entrada em paralelo.
- RNN: Tipo de rede neural onde as saídas de passos anteriores são colocadas como entradas nos passos presentes. O texto de entrada é colocado no encoder que o processa sequencialmente. Por exemplo, o encondor lê cada palavra por vez “I” “love” “dogs” e a cada palavra a RNN atualiza seu hidden state (estado oculto que carrega informações do que foi lido anteriormente), o decoder recebe o estado final (representa o máximo possível dos significado da frase inteira “I love dogs” ele é uma representação numérica do contexto/significado da frase) como ponto de partida e começa a tradução uma palavra por vez, nele também tem o hidden state (a cada palavra que ele produz ele atualiza seu próprio estado para lembrar o contexto do que já foi produzido) depois de produzir “Eu” ele precisa saber que a próxima palavra provavelmente deve continuar a frase “amo”. Basicamente o encoder processa o texto de entrada inteiro em hidden states (memory cell ou pode-se pensar neles como embedding vector) e o decoder pega esses hidden states e produz a saída. O problema do RNN era frases longas porque colocar toda a informação em um hidden atual é muita compressão, o RNN não consegue acessar diretamente os estados ocultos (hidden states) anteriores e algumas informações importantes da frase original podem ser perdidas. E para resolver esses problemas entra os mecanismos de atenção.
- Pesquisadores desenvolveram o Bahdanau attention que modificou os encoder-decoder RNN tanto que o decoder pode acessar seletivamente diferentes partes da sequência de entrada em cada passo do decoding. Com o attention o decoder pode olhar para todas as palavras da frase original e escolher quais são mais importantes naquele momento, digamos que tem a frase “I eat an pineapple” e o decoder está tentando gerar “abacaxi”, ele olha para cada palavra e vê

seu attention weights (a que tem mais peso é a mais importante). É importante notar que depois foi descoberto que não é necessário arquiteturas RNN para construir redes neurais profundas para processar a linguagem natural (chega a proposta dos transformers que inclui um mecanismo de autoatenção inspirado no Bahdanau).

- Self-attention é um mecanismo que permite cada posição da sequência de entrada considerar a relevância de todas as outras posições na mesma sequência e isso é a chave das LLMs atuais baseadas na arquitetura transformer.
- O que é atenção: “O cachorro correu porque ele estava com sono”, quando o modelo lê “ele” o mesmo precisa descobrir a quem se refere, então ele olha para outras palavras da mesma frase e verifica quais têm relação com “ele” (peso maior). Isso é attention, calcular quanto cada palavra deve prestar atenção às outras.
- Self-attention: A atenção acontece dentro da própria sequência de entrada, a sequência olha para ela mesma, a posição 5 (ele) pode prestar atenção na posição 2 (cachorro). Ele calcula as relações entre as diferentes partes da mesma entrada.
- A diferença para a atenção tradicional é que ela relaciona elementos da sequência de entrada com a de saída e não com ela mesma.
- Por que uma palavra vira vários números: computador não consegue trabalhar diretamente com a palavra “eu”, então ela vira embeddings [0.21, -0.73, 0.45, 0.12, ...] , os números individualmente não tem um significado humano simples e a rede neural aprendeu esses números durante treinamento, no começo os vetores são inicializados com valores sem significado útil, durante o treinamento a rede vai ajustando esses números, pode-se pensar como cada valor sendo uma característica daquela palavra para melhor contextualização por parte do computador.
- Depois que transformado cada palavra em embeddings temos cada palavra como $x(N) = [\dots \dots \dots]$. A self-attention pega cada palavra e cria uma nova representação dela levando em consideração as outras palavras da frase. O modelo quer entender como tal palavra se relaciona com as outras palavras. Então ele pega a posição de uma palavra ($z(N)$) e vai perguntar o quanto deve-se considerar (peso de atenção) das outras palavras. α_{21} indica o peso da posição 1 quando esta calculando o contexto da posição 2.
- Em self-attention o objetivo é calcular os vetores de contexto para cada elemento na sequência de entrada.
- Context vector: um enriquecido vetor embedding. Contém informações sobre a posição X e todos os outros elementos de entrada. Cria enriquecidas representações de cada elemento em uma sequência de entrada ao incorporar informações de todos os outros elementos da sequência. Essencial para LLMs entenderem a relação e relevância das palavras na frase entre si.

Self-attention

- Primeiro passo de implementação da self-attention é os attentions-scores (pontuações de atenção), em uma frase comparar uma palavra com ela mesma e as outras palavras vai gerar um número para cada e esses números são o attention scores (ω). O computador pega os vetores das duas palavras e faz uma operação matemática para compará-los.
- dot products: multiplicar 2 vetores (elemento com elemento) e somar seus resultados e isso nos dá 1 attention score da comparação daquela palavra com outra. Quantifica o quão perto 2 vetores estão alinhados, quanto maior o dot product maior o grau de alinhamento/similaridade entre 2 vetores.
- O próximo passo é normalizar cada attention score, obtendo pesos de atenção que somam até 1. Isso mantém estabilidade de treinamento na LLM.
- O último passo é calcular o vetor de contexto ao multiplicar os embeddings de entrada X com seus correspondentes pesos de atenção e então somar os vetores resultantes. Assim, o vetor de contexto é a soma pesada de todos os vetores de entrada, obtido ao multiplicar cada vetor de entrada por seu peso de atenção correspondente.
- **Calcular pesos de atenção:** Depois de calcular os pesos de atenção para 1 entrada e normalizar eles, deve-se calcular para todas as outras e seus vetores de contexto. Pode-se utilizar multiplicação de matrizes ao invés de loops FOR, visto que é mais rápido.

Self-attention com pesos treináveis

- Queremos calcular vetores de contexto como somas ponderadas dos vetores de entrada específicos para um determinado elemento de entrada. A diferença com o self-attention simples é que esse utiliza matrizes de pesos que são atualizadas durante o modelo de treinamento, elas são cruciais para que o modelo (módulo de atenção dentro do modelo) consiga aprender a produzir bons vetores de contexto.
- Agora entrem Q (query), K (key) e V (value). O transformer pega cada embedding e cria 3 versões dele, uma mesma palavra vai ser transformada em 3 vetores diferentes. Ele cria cada vetor com W_q , W_k e W_v que são matrizes de pesos que o modelo aprende durante o treinamento ($x \rightarrow W_q \rightarrow Q$ onde $Q = x * W_q$ da mesma forma para K e V).
- Por exemplo, a palavra journey era $x(2) = [0.55, 0.87, 0.66]$ e o tranformer pega esse vetor e faz W_q e Q e o mesmo para K e V.
- Antes comparávamos os embeddings direto (embedding x embedding) mas agr tem alg mais flexível (query x key) o modelo aprende W_q e W_k para decidir como cada palavra deve procurar informações nas outras palavras e W_v determina qual informação será carregada depois que essa atenção for calculada.
- **Query:** que tipo de informação estou procurando. **Key:** que tipo de informação eu tenho. **Value:** qual informação eu vou entregar.

- Matrizes W_q , W_k e W_v podem transformar vetores de entrada de tamanho 3 em vetores de saída de tamanho 2 através da multiplicação de matrizes. Porém, é importante notar que em GPT models as dimensões de entrada e saída são as mesmas.
- **Parâmetros de peso X Pesos de atenção:** Os weight parameters são os valores das matrizes W_{query} , W_{key} e W_{value} que o modelo aprende durante o treinamento para transformar os embeddings em Query, Key e Value. Já os attention weights são os valores calculados durante a atenção para indicar quanto cada palavra deve contribuir para o contexto de uma determinada palavra.
- O attention score é calculado como dot produto entre query e key vetores. Ele pode aparecer como ω_{22} (attention score da palavra 2 comparando com a palavra 2).
- **Fundamentação do scale-dot product attention:** A ideia é evitar que os valores do dot product fiquem grandes demais. Em modelos reais, os vetores podem ter mais de 1.000 números. Ao multiplicar e somar tantos números, o dot product pode ficar muito grande. Quando esses valores grandes passam pelo softmax, alguns pesos ficam próximos de 1 e outros de 0, fazendo os gradientes ficarem muito pequenos. Gradientes muito pequenos fazem o modelo aprender muito devagar ou até parar de aprender. Por isso, antes do softmax, dividimos os scores por $\sqrt{d_k}$. É daí que vem o nome scaled dot-product attention: dot product escalado.
- O último passo é multiplicar cada vetor value com seu respectivo peso de atenção e somar eles para obter o vetor de contexto.
- Os nomes Query, Key e Value vêm da ideia de pesquisar informações em um banco de dados. A **Query** é o que estamos procurando ou tentando entender; a **Key** funciona como uma identificação usada para descobrir quais informações são relevantes; e o **Value** é a informação que será realmente usada. Na atenção, a Query de uma palavra é comparada com as Keys das outras palavras para descobrir onde prestar atenção. Depois, os Values das palavras mais relevantes são usados para criar o novo contexto.
- Durante o treinamento, o modelo (ao ser exposto a mais dados) ajusta W_q , W_k e W_v para aprender quais palavras devem prestar atenção em quais outras.
- Em vez de criar as matrizes de pesos manualmente com `nn.Parameter(torch.rand(...))`, podemos usar `nn.Linear`. Ele já faz a multiplicação da entrada pelos pesos e também cria esses pesos de uma forma mais adequada para redes neurais. Isso ajuda o treinamento a ser mais estável. O `bias=False` significa que não queremos usar o termo extra chamado bias.
- Causal Attention impede o modelo de olhar para palavras que vêm depois. Assim, ao prever uma palavra, ele só pode usar as palavras anteriores. Multi-Head Attention divide a atenção em várias partes (heads), permitindo que cada uma aprenda diferentes relações entre as palavras.

- **Causal attention/ masked attention:** forma especializada de self-attention onde faz com que o modelo apenas considere entradas passadas e atuais em uma sequência quando calculando os attention scores. Esse é um contraste com o self-attention padrão (permite acesso a sequência toda de entrada de uma só vez). Para cada token processado nós mascaramos os tokens futuros (o que vem em seguida do token atual), mascaramos os pesos de atenção dos tokens futuros e fazemos com que os pesos de atenção que aparecem sejam normalizados e somem 1 em cada linha. Pesos acima da diagonal são zerados.
- **Information leakage:** Quando usamos uma máscara na atenção, queremos impedir que um token atual consiga olhar para tokens futuros. Pode parecer que isso não funciona, porque, antes de aplicar a máscara, todos os tokens participam do cálculo do softmax, inclusive os tokens que serão mascarados. Porém, depois que aplicamos a máscara, os valores das posições bloqueadas deixam de participar efetivamente do cálculo da atenção. Quando fazemos a renormalização dos pesos, estamos basicamente calculando o softmax novamente considerando apenas os tokens que não foram mascarados. Assim, um token só recebe atenção dos tokens que estão permitidos, e nenhuma informação dos tokens futuros consegue influenciá-lo. Isso é justamente o que garante que não exista information leakage (vazamento de informação) durante o treinamento do modelo.
- Um jeito de obter a matriz de peso de atenção mascarada é mascarar os attention scores com valores negativos infinitos antes de aplicar o softmax, porque o softmax trata esse valor como 0.
- **Overfitting:** Acontece quando o modelo decora muito bem os dados de treinamento, mas tem dificuldade para lidar com dados novos. Não aprende a generalizar.
- **Dropout:** Técnica usada durante o treinamento para desligar aleatoriamente algumas partes do modelo. Isso ajuda a evitar que o modelo fique dependente demais de determinadas informações e reduz o risco de overfitting. No Transformer, o dropout também pode ser aplicado na atenção. Nesse caso, depois de calcular os pesos de atenção, alguns desses pesos são escolhidos aleatoriamente e zerados. Assim, o modelo aprende a não depender sempre dos mesmos pesos de atenção. O dropout é usado somente durante o treinamento e é desativado quando o modelo está sendo utilizado normalmente.

Implementação de elementos multi-heads

- Implementa vários mecanismos de causal attention em paralelo.
- **Multi-head attention:** Dividir o mecanismo de atenção em múltiplas “heads”, cada uma operando independentemente. Criar múltiplas instâncias do mecanismo de self-attention, cada um com seus pesos, e combinar suas saídas. Isso pode ser intenso em termos de computação, mas é crucial para o tipo de complexidade nos padrões de reconhecimento de modelos LLMs. Ao invés de 1 query, key e Value, agora temos Q2, K2 e V2 e 2 heads geram 2 matrizes de

pesos de atenção, incluindo causal attention e dropout máscaras. Podemos combinar o Z1 e Z2 em um único vetor de contexto.

- A diferença principal é como as várias heads de atenção são organizadas. No `MultiHeadAttentionWrapper`, cada head é criada separadamente como um objeto `CausalAttention`, e depois os resultados de todas as heads são juntados (concatenados). Já na classe `MultiHeadAttention`, todas as heads ficam dentro da mesma classe, os dados são divididos em várias heads usando `reshape`, cada uma calcula a atenção e, no final, os resultados são reunidos novamente. Em resumo, a primeira abordagem cria vários objetos de atenção, enquanto a segunda faz tudo dentro de um único objeto, sendo mais eficiente e organizada, ela começa com multi-head camada e então internamente divide a camada em attention heads individuais.
- **Exemplo:** vetor = [10, 20, 30, 40] e queremos 2 heads de atenção, head1 [10, 20] e head2 [30, 40] assim $d_{out} = 4$, $num_heads = 2$ e $head_dim = 4 / 2 = 2$. Então cada cabeça fica responsável por 2 números (`.view()` é responsável por isso) então `view(1, 3, 2, 4) = view(1, 3, 2, 4)` tokens, 2 heads, 2 dimensões nas heads. O `transpose()` troca a posição de 2 dimensões para deixar os dados organizados de maneira que facilite o cálculo da atenção, `transpose(1, 2)` troca a posição de `num_tokens` e `num_heads` então antes era (1, 3, 2, 4) agr fica (1, 3, 4, 2) e comparamos cada token com cada outro token
- **Transpose:** Uma forma de fazer várias multiplicações de matrizes ao mesmo tempo (uma para cada head).